ISE 218: Fundamentals of Information Technology

Practice Problems for Unit 5: Memory

Directions: Below is a set of exercises and problems you should do for practice. Answering this set of questions should be just one part of your preparation for quizzes and exams. In addition to solving these problems you should:

- Review your homework assignments.
- Review the lecture notes and read the relevant sections of the course textbook.
- Do additional practice problems from the textbook; see the problems at the end of each chapter.
- Form a study group and test each other on the material.

1. Suppose a computer using direct mapped cache has 2^{20} bytes of byte-addressable main memory, and a cache of 32 blocks, where each cache block contains 512 bytes.

   a. How many blocks of main memory are there? Express your answer in decimal in 2^n form.

      $2^{20}/2^9 = 2^{11}$ or 2048 blocks

   b. What is the format of a memory address as seen by the cache, i.e., what are the sizes of the tag, block, and offset fields?

      32 blocks → 5 bits for the block field

      512 bytes per block → 9 bits for the offset field

      $20-5-9 = 6$ bits for the tag field

   c. To which cache block will the memory address 0x2ED5A map? Express your answer in decimal.

      2ED5A = 00101110110101011010 = 001011 **10110** 101011010, which implies Block 10110_2, which is 22_{10}.

2. Suppose a computer using direct mapped cache has 2^{24} bytes of byte-addressable main memory, and a cache of 128 blocks, where each cache block contains 64 bytes.

   a. How many blocks of main memory are there? Express your answer in decimal.

      $2^{24}/2^6 = 2^{18}$ blocks of main memory

   b. What is the format of a memory address as seen by the cache, i.e., what are the sizes of the tag, block, and offset fields? Express your answers in decimal.

      128 blocks → 7 bits for the block field

      64 bytes per block → 6 bits for the offset field

      $24-7-6 = 11$ bits for the tag field

c. To which cache block will the memory address 0x249A03 map? Express your answer in decimal.

249A03 = 00100100100 1101000 000011, which implies block # (64+32+8) = 104_{10}.

3. Suppose a computer using direct mapped cache has 2^{20} bytes of byte-addressable main memory, and a cache of 64 blocks, where each cache block contains 256 bytes.

a. How many blocks of main memory are there? Express your answer in decimal.

$2^{20}/2^8 = 2^{12}$ blocks of main memory

b. What is the format of a memory address as seen by the cache, i.e., what are the sizes of the tag, block, and offset fields? Express your answers in decimal.

64 blocks → 6 bits for the block field

256 bytes per block → 8 bits for the offset field

$20-6-8 = 6$ bits for the tag field

c. To which cache block will the memory address 0xA9CBA map? Express your answer in decimal.

A9CBA = 1010 1001 1100 1011 1010 = 101010 011100 10111010, which implies block # (16+8+4) = 28_{10}.

4. Suppose a computer using direct mapped cache has 2^{24} bytes of byte-addressable main memory, and a cache of 128 blocks, where each cache block contains 2048 bytes.

a. How many blocks of main memory are there? Express your answer in decimal in 2^n form.

$2_{24}/2_{11} = 2_{13}$

b. What is the format of a memory address as seen by the cache, i.e., what are the sizes of the tag, block, and offset fields?

128 blocks → 7 bits for the block field

2048 bytes per block → 11 bits for the offset field

$24-7-11 = 6$ bits for the tag field

c. To which cache block will the memory address 0xA44E2B map? Express your answer in decimal.

A44E2B = 1010 0100 0100 1110 0010 1011 = 101001 **0001001** 11000101011, which implies Block 0001001_2, which is 9_{10}.

5. Suppose a computer using fully associative cache has 2^{16} bytes of byte-addressable main memory and a cache of 64 blocks, where each cache block contains 128 bytes.

a. How many blocks of main memory are there? Express your answer in decimal.

$2^{16}/2^7 = 2^9$ or 512 blocks

b. What is the format of a memory address as seen by the cache, i.e., what are the sizes of the tag and offset fields?

128 bytes per block → 7 bits for the offset field

16−7 = 9 bits for the tag field

6. Suppose a computer using fully associative cache has 2^{24} bytes of byte-addressable main memory and a cache of 128 blocks, where each cache block contains 512 bytes. What is the tag value (in decimal) of the byte stored at physical memory address 0x00099A?

   Address format: offset field is 9 bits, leaving 15 bits for tag.

   0000 0000 0000 1001 1001 1010 000000000000100 110011010

   $100_2 = 4_{10}$

7. Suppose a computer using fully associative cache has 2^{16} bytes of byte-addressable main memory and a cache of 32 blocks, where each cache block contains 64 bytes.

   a. How many blocks of main memory are there? Express your answer in decimal.

      $2^{16}/2^6 = 2^{10}$ or 1024 blocks

   b. What is the format of a memory address as seen by the cache, i.e., what are the sizes of the tag and offset fields?

      64 bytes per block → 6 bits for the offset field

      16−6 = 10 bits for the tag field

8. Suppose a computer using fully associative cache has 2^{16} bytes of byte-addressable main memory and a cache of 1024 blocks, where each cache block contains 32 bytes. What is the tag value (in decimal) of the byte stored at physical memory address 0x04B7?

   Address format: offset field is 5 bits, leaving 11 bits for tag.

   0000 0100 1011 0111 00000100101 10111

   $100101_2 = 32+4+1 = 37_{10}$

9. Suppose you have a byte-addressable virtual address memory system with 2048 virtual pages of 512 bytes each and 256 page frames.

   a. How many bits are in a virtual address? Express your answer in decimal.

      $2^{11} \times 2^9 = 2^{20}$, so 20 bits

   b. What is the size of physical memory in KB? Express your answer in decimal.

      $2^8 \times 2^9 = 2^{17} = 2^7 \times 2^{10}$ bytes = 2^7 KB or 128 KB

10. Suppose you have a byte-addressable virtual address memory system with 512 virtual pages of 1024 bytes each and 64 page frames.

    a. How many bits are in a virtual address? Express your answer in decimal.

       $2^9 \times 2^{10} = 2^{19}$, so 19 bits

b. What is the size of physical memory in KB? Express your answer in decimal.

$2^6 \times 2^{10} = 2^{16} = 2^6 \times 2^{10}$ bytes = 2^6 KB or 64 KB

11. Suppose you have a byte-addressable virtual address memory system with 8 virtual pages of 4096 bytes each, and 4 page frames. Given the following page table, answer the questions below.

Page	Frame	Valid Bit
0	1	0
1	3	0
2	3	1
3	0	0
4	1	1
5	3	0
6	2	1
7	2	0

a. How many bits are in a virtual address?

Virtual memory size: $8 \times 4096 = 2^3 \times 2^{12} = 2^{15}$. Thus, 15 bits are needed.

b. How many bits are in a physical address?

Physical memory size: $4 \times 4096 = 2^2 \times 2^{12} = 2^{14}$. Thus, 14 bits are needed.

c. What physical addresses correspond to the following virtual addresses? If a reference to a virtual address causes a page fault, simply indicate this is the case. Give your answers in hexadecimal. i. 0x3F90

011 1111 1001 0000_2 is the virtual memory address in binary. Page 3 is not loaded, so this memory reference causes a page fault.

ii. 0x6570

110 0101 0111 0000_2 is mapped to 10 0101 0111 0000_2, which is 0x2570.

d. How many entries are in the page table for a process that uses exactly half of virtual memory? (Think carefully before you write your answer!)

The page table must have an entry for EVERY virtual page, so it must have 8 entries.

12. Suppose you have a byte-addressable virtual address memory system with 8 virtual pages of 256 bytes each, and 4 page frames. Given the following page table, answer the questions below.

Page	Frame	Valid Bit
0	1	1
1	3	0

2	3	1
3	0	1
4	1	0
5	3	0
6	2	0
7	2	0

a. How many bits are in a virtual address?

Virtual memory size: $8 \times 256 = 2^3 \times 2^8 = 2^{11}$. Thus, 11 bits are needed.

b. How many bits are in a physical address?

Physical memory size: $4 \times 256 = 2^2 \times 2^8 = 2^{10}$. Thus, 10 bits are needed.

c. How many entries are in the page table for a process that uses exactly half of virtual memory? (Think carefully before you write your answer!)

The page table must have an entry for EVERY virtual page, so it must have 8 entries.

d. What physical addresses correspond to the following virtual addresses? If a reference to a virtual address causes a page fault, simply indicate this is the case. Give your answers in hexadecimal. i. 0x28E

$010\ 1000\ 1110_2$ is mapped to $11\ 1000\ 1110_2$, which is 0x38E.

ii. 0x748

$110\ 0100\ 1000_2$ is the virtual memory address in binary. Page 7 is not loaded, so this memory reference causes a page fault.

13. Consider a system with a main memory access time of 200 ns supported by a cache having a 30 ns access time and a hit rate of 90%. What is the effective access time (EAT) in nanoseconds if:

a. access to cache and main memory occur *concurrently*? Simplify your answer as much as possible.

$EAT = 0.9(30\text{ns}) + 0.1(200\text{ns}) = 27\text{ns} + 20\text{ns} = 47\text{ns}$

b. access to cache and main memory occur *sequentially*? Simplify your answer as much as possible.

$EAT = 0.9(30\text{ns}) + 0.1(200\text{ns} + 30\text{ns}) = 27\text{ns} + 0.1(230\text{ns}) = 27\text{ns} + 23\text{ns} = 50\text{ns}$

14. Consider a system with a main memory access time of 400 ns supported by a cache having a 20 ns access time and a hit rate of 90%. What is the effective access time (EAT) in nanoseconds if:

a. access to cache and main memory occur *concurrently*? Simplify your answer as much as possible.

$EAT = 0.9(20\text{ns}) + 0.1(400\text{ns}) = 18\text{ns} + 40\text{ns} = 58\text{ns}$

b. access to cache and main memory occur *sequentially*? Simplify your answer as much as possible.

$EAT = 0.9(20ns)+0.1(400ns+20ns) = 18ns+0.1(420ns) = 18ns+42ns = 60ns$

15. Suppose a main memory access takes 100 ns, the page fault rate is 20%, and it takes 40 ms to load a page from disk. What is the expected access time in *nanoseconds* assuming we ignore the influence of cache memory? Simplify your answer as much as possible.

    $EAT = 0.80(100ns+100ns)+0.2(40ms) = 0.8(200ns)+0.2(40{,}000{,}000ns) = 160ns+8{,}000{,}000ns = 8{,}000{,}160ns$

16. Suppose a main memory access takes 80 ns, the page fault rate is 25%, and it takes 10 ms to load a page from disk. What is the expected access time in *nanoseconds* assuming we ignore the influence of cache memory? Simplify your answer as much as possible.

    $EAT = 0.75(80ns+80ns)+0.25(10ms) = 0.75(160ns)+0.25(10{,}000{,}000ns) = 120ns+2{,}500{,}000ns = 2{,}500{,}120ns$

17. Suppose a main memory access takes 50 ns, the page fault rate is 20%, and it takes 30 ms to load a page from disk. What is the expected access time in nanoseconds assuming we ignore the influence of cache memory?

    $EAT = 0.8(50\text{ ns} + 50\text{ ns}) + 0.2(30ms) = 0.8(100\text{ ns}) + 0.2(30000000\text{ ns}) = 80\text{ ns} + 6000000\text{ ns} = 6000080ns$

18. Suppose a main memory access takes 20 ns, the page fault rate is 30%, and it takes 5 ms to load a page from disk. What is the expected access time in nanoseconds assuming we ignore the influence of cache memory?
    $EAT = 0.7(20ns+20ns)+0.3(5ms) = 0.7(40ns)+0.3(5000000ns) = 28ns+1500000ns = 1500028ns$

19. Suppose a program makes a sequence of memory references that cause the following memory blocks to be read, in this order: A, D, E, D, C, A, E, B. The CPU's cache has 3 blocks and uses FIFO as the replacement policy. Show the state of the cache after each memory block access. Assume the left-hand side of the queue is the head of the queue. Use an arrow to mark the oldest block in the queue.

    A _ _ reference A
    ^

    A D _ reference D
    ^

    A D E reference E
    ^

    A D E reference D
    ^

    C D E reference C
    ^

C A E reference A
^

C A E reference E
^

C A B reference B
^

20. Suppose a program makes a sequence of memory references that cause the following memory blocks to be read, in this order: D, B, C, D, A, C, E, D. The CPU's cache has 3 blocks and uses FIFO as the replacement policy. Show the state of the cache after each memory block access. Assume the left-hand side of the queue is the head of the queue. Use an arrow to mark the oldest block in the queue.

D _ _ reference D
^

D B _ reference B
^

D B C reference C
^

D B C reference D
^

A B C reference A
^

A B C reference C
^

A E C reference E
^

A E D reference D
^